
GPU-Accelerated Monte Carlo Simulation for Financial Option Pricing

Talha Sarlık & Mehmet Bedirhan Önder
704241052 · 704251026

Summary

We are going to implement a GPU-accelerated Monte Carlo simulation for pricing European options using the Black-Scholes model on NVIDIA GPUs with CUDA. We will start with a naive parallel implementation and progressively optimize it using shared memory, warp-level intrinsics, and variance reduction techniques. The main goal is to do a detailed performance analysis comparing different implementation strategies against each other and against a CPU baseline.

Background

Monte Carlo simulation is a technique for estimating quantities that are hard to compute analytically, by running a large number of random experiments and averaging the results. In finance, it is commonly used for option pricing: to find the fair price of a contract today, you simulate many possible futures of the underlying asset and average the resulting payoffs.

In the Black-Scholes Monte Carlo framework, the asset price follows a Geometric Brownian Motion (GBM). Each simulated price path is generated step-by-step as:

$$S(t + dt) = S(t) * \exp((r - \sigma^2/2)*dt + \sigma*\sqrt{dt}*Z)$$

where r is the risk-free rate, σ is the volatility, and Z is drawn from a standard normal distribution. After simulating N paths, the option price is the discounted average of the payoffs at maturity (e.g., $\max(S_T - K, 0)$ for a European call). The key observation is that each path is fully independent from the others — there are no data dependencies between paths — which makes this a very good fit for GPU parallelization.

The Challenge

Even though the problem is embarrassingly parallel at the path level, there are several things that make it non-trivial to implement efficiently on a GPU:

- Random number generation: Each thread needs its own independent, high-quality random number stream. If all threads share the same seed or use correlated streams, the simulation results become statistically biased. Managing per-thread cuRAND states has a meaningful memory and initialization cost, and different generator types

(XORWOW, MRG32k3a, Philox) have different speed and quality trade-offs worth analyzing.

- Memory access patterns: Each price path is a sequential computation — step $t+1$ depends on step t — so there is no intra-thread parallelism within a path. The challenge is making sure that the writes to the output array are coalesced across threads to keep memory bandwidth high.
- Reduction: After all paths are simulated, we need to sum millions of payoff values into a single average. This requires an efficient parallel reduction. Doing it naively with global atomics is slow. We will implement a shared memory tree reduction and compare it with warp shuffle (`__shfl_down_sync`) based approaches.
- Variance reduction: Standard Monte Carlo converges at $O(1/\sqrt{N})$, which means you need 100x more samples to get 10x more precision. Antithetic variates is a technique that pairs each sample Z with $-Z$ to reduce variance without extra simulation cost, but it introduces a dependency between thread pairs, which complicates the straightforward parallelization.

Resources

We will use Google Colab for development and benchmarking, which gives us access to NVIDIA T4 and A100 GPUs. CUDA kernels will be written in CUDA C/C++ and compiled with `nvcc`. For random number generation we will use the cuRAND device API. For profiling we will use NVIDIA Nsight Compute and Nsight Systems. The CPU reference implementation will be written in C. Python and Matplotlib will be used for post-processing results and generating plots.

We are starting from scratch — we will not be using any existing CUDA Monte Carlo codebase. Main references: NVIDIA cuRAND Library User Guide, CUDA C Best Practices Guide, and P. Glasserman, Monte Carlo Methods in Financial Engineering (Springer, 2003).

Goals and Deliverables

Plan to achieve:

- A working CPU baseline implementation in C for correctness checking.
- A GPU kernel where each thread simulates one full price path using cuRAND.
- A parallel reduction kernel using shared memory to compute the final average payoff.
- A comparison of different cuRAND generator types in terms of throughput and sample quality.
- A benchmark sweeping number of paths (10K to 100M) and measuring speedup over CPU, pricing error vs. Black-Scholes closed form, and memory bandwidth utilization.
- Nsight Compute profiling reports for each kernel version showing occupancy, warp efficiency, and memory throughput.

Hope to achieve:

- Antithetic variates on GPU: pair each thread with its antithetic counterpart and measure the improvement in convergence rate (error vs. number of paths).
- CUDA streams to overlap memory transfers and kernel execution via double buffering.
- Extension to Asian options, which require averaging the price at each time step — this introduces more work per thread and changes the memory access pattern.

Poster session demo:

We plan to have a live interactive demo where the audience can change the number of paths and immediately see the option price converge and the runtime update on screen. We will also show speedup graphs (CPU vs. GPU across different path counts), convergence plots comparing naive and antithetic Monte Carlo, and Nsight timeline screenshots showing the kernel execution. We are targeting around 200x speedup over single-threaded CPU at 10M+ paths.

Platform Choice

This problem is a natural fit for GPU because the parallelism is at the path level and paths are fully independent — there is no communication between them at all. With 10 million paths, we can launch 10 million threads and have each one do its own simulation. A CPU with 16 cores would need to do the same work sequentially within each core, which is orders of magnitude slower.

Beyond the raw thread count, GPUs also have much higher memory bandwidth than CPUs (e.g., ~600 GB/s on A100 vs ~50 GB/s for a CPU), which matters when reading and writing millions of float values. The cuRAND library also provides high-quality random number generation directly on the GPU without needing to transfer data from the CPU. For these reasons, CUDA on an NVIDIA GPU is the right platform for this workload.